

MANUAL DE USUARIO

Validador de Cadenas — Autómata KRAVNIKA

Aplicación de escritorio para validar cadenas de texto según la gramática del lenguaje construido Kravnika

1. Introducción

Este programa es una herramienta de escritorio que permite escribir una cadena de texto y verificar si cumple con las reglas gramaticales del lenguaje construido “Kravnika”. El sistema está compuesto por tres partes que trabajan juntas:

- Una interfaz gráfica (ventana) donde el usuario escribe la cadena y ve el resultado.
- Un motor de validación (autómata) escrito en C++, que aplica las reglas de la gramática Kravnika.
- Una librería de conectores lógicos (AND, OR, NOT, XOR, IMPLIES, IFF) que el motor usa internamente para combinar las distintas condiciones de validez.

El usuario final solo necesita usar la ventana: escribir una cadena, presionar “Validar” y leer el resultado. Este manual explica tanto el uso de la interfaz como las reglas que determinan si una cadena es válida.

2. Componentes del sistema

Archivo / carpeta	Función
ui.py (interfaz)	Ventana de la aplicación (Tkinter). Recibe la cadena, la envía al motor y muestra el resultado y el historial.
Automate/automate.cpp	Código fuente del autómata que valida la cadena según la gramática Kravnika.
Automate/logical-connector-lib.cpp	Librería con los conectores lógicos (AND, OR, NOT, XOR, IMPLIES, IFF) usados por el autómata.
Automate/output.exe	Ejecutable compilado a partir de los dos archivos anteriores. Es el que la interfaz llama para validar.
kravnika_creator.png	Imagen decorativa mostrada en la ventana.
Fuente Kravnika (.otf / .ttf)	Tipografía personalizada usada para mostrar texto en “alfabeto Kravnika”.

Nota: Los nombres exactos de los archivos pueden variar ligeramente según cómo se hayan guardado en tu proyecto, pero la estructura de carpetas debe respetarse: el ejecutable debe estar en una subcarpeta llamada “Automate” junto al script de la interfaz.

3. Requisitos e instalación

3.1 Software necesario

- Python 3, con las librerías tkinter (incluida normalmente con Python) y Pillow (PIL) instalada: `pip install pillow`
- Un compilador de C++ (por ejemplo g++) para generar el ejecutable del autómata.

3.2 Compilar el motor de validación

Antes de usar la interfaz por primera vez, hay que compilar el autómata en un ejecutable llamado `output.exe` dentro de la carpeta `Automate`. Ejemplo de compilación con g++:

```
g++ automate.cpp -o Automate/output.exe
```

Nota: El código incluye el encabezado “logical-connector-lib.h”. Asegúrate de que ese archivo (la versión .h de la librería de conectores lógicos) esté disponible junto a `automate.cpp` al compilar.

3.3 Estructura de carpetas esperada

```
proyecto/  ui.py  kravnika_creator.png  Automate/  automate.cpp  logical-connector-lib.cpp  logical-connector-lib.h  output.exe
```

3.4 Instalar la fuente Kravnika

Para que el texto se muestre correctamente con la tipografía personalizada, instala en el sistema operativo el archivo `Kravnika-Regular.otf` o `Kravnika-Regular.ttf` (doble clic sobre el archivo y elegir “Instalar”, o copiarlo a la carpeta de fuentes del sistema). Si la fuente no está instalada, la interfaz mostrará el texto con una fuente alternativa del sistema.

4. Iniciar la aplicación

Con el ejecutable ya compilado y la fuente instalada, abre una terminal en la carpeta del proyecto y ejecuta:

```
python ui.py
```

Se abrirá la ventana “Validador de Cadenas - Autómata KRAVNIKA”, lista para usarse.

5. Descripción de la interfaz

Elemento	Descripción
Título / logo	Nombre de la aplicación en la tipografía Kravnika e imagen decorativa.
Campo “Cadena a validar”	Casilla de texto donde se escribe la cadena que se quiere comprobar.
Botón “Validar”	Envía la cadena escrita al motor de validación. También se activa presionando Enter dentro del campo de texto.
Etiqueta de resultado	Muestra  si la cadena es válida o  si no lo es, justo debajo del campo de entrada.
Tabla “Historial”	Lista todas las cadenas evaluadas en la sesión actual, coloreadas en verde (válida) o rojo (inválida).

Elemento	Descripción
Botón de idioma (Español / Kravnika)	Alterna cómo se muestra el texto del historial: en el alfabeto original o en una versión adaptada al español.
Botón “Limpiar historial”	Borra todas las cadenas registradas en el historial, previa confirmación.

6. Cómo validar una cadena — paso a paso

- Escribe la cadena que quieras comprobar en el campo “Cadena a validar”.
- Presiona el botón “Validar” o la tecla Enter.
- La aplicación envía la cadena al ejecutable del autómata y espera su respuesta (máximo 5 segundos).
- El resultado aparece debajo del campo:  La cadena es válida, o  La cadena no es válida.
- La cadena se agrega automáticamente al historial, junto con su resultado.
- El campo de texto se limpia y queda listo para la siguiente cadena.

7. Reglas para que una cadena sea válida en Kravnika

Una cadena se evalúa en tres niveles: la cadena completa, las frases que la componen y las palabras dentro de cada frase. Para que la cadena resulte válida, todos los niveles deben cumplirse.

7.1 Frases

- La cadena completa debe estar dividida en una o más frases, y cada frase debe terminar en un punto (.).
- Si el texto no termina en un punto, la cadena se considera inválida.
- Dentro de cada frase, los signos ¿ ? y ¡ ! son opcionales, pero si se usan deben aparecer en pares (abrir y cerrar). Un signo de interrogación o exclamación “suelto” (en cantidad impar) invalida la frase.

7.2 Palabras

- Dentro de una frase, las palabras se separan con dos puntos (:).
- Cada palabra debe cumplir, al mismo tiempo, tres condiciones: marca de mayúscula, ausencia de nombres prohibidos y formato numérico correcto (ver más abajo).

7.3 Marca de “mayúscula” (comilla o guion)

Como el alfabeto Kravnika no distingue mayúsculas de forma visual, la primera letra “en mayúscula” de cada palabra se marca con un símbolo justo después de la primera letra:

- Comilla (") inmediatamente después de la primera letra — por ejemplo: N"ACI
- Guion (-) inmediatamente después de la primera letra — por ejemplo: H-OLA

Cada palabra debe usar exactamente uno de los dos marcadores (nunca ambos, nunca ninguno), y ese marcador no debe repetirse en el resto de la palabra.

7.4 Nombres prohibidos

Ninguna palabra puede contener, en cualquier parte de su texto (sin importar mayúsculas, comillas, guiones o números), alguno de estos nombres:

- jorge

- jhonathan
- fabritzio
- rodrigo

Nota: La comprobación es por coincidencia de subcadena: si cualquiera de estos nombres aparece dentro de una palabra más larga, esa palabra también se considera inválida.

7.5 Números y separadores

Los números dentro de una palabra pueden agruparse usando separadores especiales. Cada separador debe ir seguido de un dígito del 1 al 9:

Separador	Uso
>;	Separador de miles
;	Separador de centenas
	Separador de decimales

Si una palabra no usa ninguno de estos separadores, esta regla se cumple automáticamente. Ejemplo de número válido dentro de una palabra: A"NIO|>;27+|>;5||12.

7.6 Ejemplo comentado

Tomando como referencia el ejemplo incluido en el código fuente del autómata:

```
N"ACI:E"L:A"NIO|>;27+|>;5||12. H-OLA:ESTO:E-S:UNA:P"RUE"BA.
ES"TA:E-"S:L"A:T-"ERCERA:FR|>;51ASE. M-e:l"lamo:rodrigo235.
```

- Las tres primeras frases combinan palabras con comilla y con guion como marca de mayúscula, además de un número con separadores.
- La última frase (“M-e:l"lamo:rodrigo235.”) resulta inválida porque la palabra “rodrigo235” contiene el nombre prohibido “rodrigo”.

8. Historial de validaciones

Cada cadena evaluada se agrega a la tabla de historial, mostrando el texto en verde si fue válida o en rojo si fue inválida.

- Botón de idioma: cambia la fuente y el formato del texto mostrado en el historial, alternando entre la visualización “Kravnika” y una versión “Español” simplificada (sin comillas y con guiones bajos en lugar de dos puntos).
- Botón “Limpiar historial”: pide confirmación y, si se acepta, borra todas las cadenas registradas y el resultado mostrado en pantalla.

Nota: El historial se guarda solo en memoria mientras la ventana está abierta; al cerrar la aplicación se pierde.

9. Solución de problemas comunes

Mensaje / situación	Causa probable y solución
“No se encontró el ejecutable en ...”	El archivo Automate/output.exe no existe o está en otra ruta. Compila automate.cpp como se indica en la sección 3.2 y verifica que quede dentro de la carpeta Automate.

Mensaje / situación	Causa probable y solución
“El programa tardó demasiado en responder.”	El motor de validación no respondió en 5 segundos. Revisa que el programa no quede esperando otra entrada o en un bucle infinito.
“(salida no reconocida) ...”	El ejecutable devolvió un texto distinto a “1” o “0”. Puede deberse a mensajes de depuración adicionales impresos por el autómata (ver nota técnica en la sección 10).
El texto no se ve con la tipografía Kravnika	La fuente no está instalada en el sistema. Instala Kravnika-Regular.otf o .ttf y reinicia la aplicación.
No aparece la imagen decorativa	Verifica que kravnika_creator.png esté en la misma carpeta que ui.py.

10. Notas técnicas y limitaciones conocidas

- El motor de validación (automate.cpp) imprime en consola varios mensajes de depuración (por ejemplo el contenido de cada palabra y el resultado de cada condición) además del veredicto final. La interfaz espera que la salida sea exactamente “1” o “0”; si se agregan o mantienen mensajes adicionales, puede aparecer “(salida no reconocida)” aunque la cadena sí haya sido evaluada correctamente.
- Para evitar ese problema, se recomienda que el autómata imprima únicamente “1” o “0” al final de su ejecución (por ejemplo con `cout << aut->get_result();`), dejando los mensajes de depuración solo para pruebas internas.
- La comprobación de nombres prohibidos no distingue palabras completas: detecta el nombre en cualquier posición dentro de la palabra.
- La regla de los signos ¿ ? y ¡ ! se cumple con que uno de los dos aparezca un número par de veces (incluido cero); no es necesario usar ambos signos en la misma frase.